SECTION -A

Q 1: 20 friends put their wallets in a row. The first wallet contains 20 dollars, the second has 30 dollars, the third has 40 dollars, and so on, with each wallet having 10 dollars more than the previous one. Since the data is already sorted in ascending order, no sorting is required. But if you are given a chance to sort the wallets, which sorting technique would be best to apply? Write a C++ program to implement your chosen sorting approach.

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements that are greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
```

```
    int wallets[20];

    // Initializing wallets with values 20, 30, 40 ... 210
    for (int i = 0; i < 20; i++) {
        wallets[i] = 20 + (i * 10);
    }

    cout << "Wallets before sorting:\n";
    for (int i = 0; i < 20; i++) {
        cout << wallets[i] << " ";
    }
    cout << "\n";

    // Applying insertion sort
    insertionSort(wallets, 20);

    cout << "Wallets after sorting:\n";
    for (int i = 0; i < 20; i++) {
        cout << wallets[i] << " ";
    }
    cout << "\n";

    return 0;
}
```

Q 2: In a park, 10 friends were discussing a game based on sorting. They placed their wallets in a row. The maximum money in any wallet is **$6**. Among them, **3 wallets contain exactly $2**, **2 wallets contain exactly $3**, **2 wallets are empty ($0)**, **1 wallet contains $1**, and **1 wallet contains $4**. Which sorting technique would you apply to sort the wallets on the basis of the money they contain? Write a program to implement your chosen sorting technique.

```
#include <iostream>
using namespace std;

void countingSort(int arr[], int n, int maxValue) {
    int count[maxValue + 1] = {0};

    // Step 1: Count frequency
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // Step 2: Reconstruct sorted array
    int index = 0;
    for (int i = 0; i <= maxValue; i++) {
        while (count[i] > 0) {
            arr[index++] = i;
            count[i]--;
        }
    }
```

```
}

int main() {
    int wallets[] = {2, 3, 0, 2, 4, 0, 2, 3, 1, 2};
    int n = 10;

    countingSort(wallets, n, 6);

    cout << "Wallets after sorting:\n";
    for (int i = 0; i < n; i++) {
        cout << wallets[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Q 3: During a college fest, 12 students participated in a gaming competition. Each student's score was recorded as follows: 45, 12, 78, 34, 23, 89, 67, 11, 90, 54, 32, 76 The organizers want to arrange the scores in **ascending order** to decide the ranking of the players. Since the data set is unsorted and contains numbers spread across a wide range, the most efficient technique to apply here is **Quick Sort**. Write a C++ program to implement **Quick Sort** to arrange the scores in ascending order.

```
#include <iostream>
using namespace std;
```

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int scores[] = {45, 12, 78, 34, 23, 89, 67, 11, 90, 54, 32, 76};
    int n = 12;
```

```
    quickSort(scores, 0, n - 1);

    cout << "Scores in ascending order:\n";
    for (int i = 0; i < n; i++) {
        cout << scores[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Q 4: A software company is tracking project deadlines (in days remaining to submit). The deadlines are: 25, 12, 45, 7, 30, 18, 40, 22, 10, 35. The manager wants to arrange the deadlines in **ascending order** to prioritize the projects with the least remaining time. For efficiency, the project manager hints to the team to apply a **divide-and-conquer technique that divides the array into unequal parts**. Write a C++ program to sort the project deadlines using the above sorting technique.

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
```

```
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int deadlines[] = {25, 12, 45, 7, 30, 18, 40, 22, 10, 35};
    int n = 10;

    quickSort(deadlines, 0, n - 1);

    cout << "Deadlines in ascending order:\n";
    for (int i = 0; i < n; i++) {
        cout << deadlines[i] << " ";
```

```
    }
    cout << endl;
    return 0;
}
```

Q 5: Suppose there is a square named **SQ-1**. By connecting the midpoints of SQ-1, we create another square named **SQ-2**. Repeating this process, we create a total of **50 squares** {SQ-1, SQ-2, …, SQ-50}. The areas of these squares are stored in an array. Your task is to **search whether a given area is present in the array or not**.What would be the best searching approach? Write a C++ program to implement this approach.

```
#include <iostream>
using namespace std;

bool binarySearch(double arr[], int n, double key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return true;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return false;
```

```
}

int main() {
    double areas[50];
    areas[0] = 100.0; // Example starting area
    for (int i = 1; i < 50; i++) {
        areas[i] = areas[i - 1] / 2.0;  // Each next square is half area
    }

    double key;
    cout << "Enter area to search: ";
    cin >> key;

    if (binarySearch(areas, 50, key))
        cout << "Area found!" << endl;
    else
        cout << "Area not found." << endl;

    return 0;
}
```

Q 6: Before a match, the chief guest wants to meet all the players. The head coach introduces the first player, then that player introduces the next player, and so on, until all players are introduced. The chief guest moves forward with each introduction, meeting the players one at a time. How would you implement the above activity using a Linked List? Write a C++ program to implement the logic.

```
#include <iostream>
using namespace std;

struct Node {
    string player;
    Node* next;
};

void introducePlayers(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << "Meeting player: " << temp->player << endl;
        temp = temp->next;
    }
}

int main() {
    Node* head = new Node{"Player 1", nullptr};
    head->next = new Node{"Player 2", nullptr};
    head->next->next = new Node{"Player 3", nullptr};
    head->next->next->next = new Node{"Player 4", nullptr};

    introducePlayers(head);

    return 0;
```

}

Q 7: A college bus travels from stop A → stop B → stop C → stop D and then returns in reverse order D → C → B → A. Model this journey using a **doubly linked list**. Write a program to:

- Store bus stops in a doubly linked list.
- Traverse forward to show the onward journey.
- Traverse backward to show the return journey.

```
#include <iostream>
using namespace std;

struct Node {
    string stop;
    Node* next;
    Node* prev;
};

void traverseForward(Node* head) {
    Node* temp = head;
    cout << "Onward Journey: ";
    while (temp != nullptr) {
        cout << temp->stop << " ";
        if (temp->next == nullptr) break;
        temp = temp->next;
    }
    cout << endl;
```

```
    cout << "Return Journey: ";
    while (temp != nullptr) {
        cout << temp->stop << " ";
        temp = temp->prev;
    }
    cout << endl;
}

int main() {
    Node* A = new Node{"A", nullptr, nullptr};
    Node* B = new Node{"B", nullptr, A};
    A->next = B;
    Node* C = new Node{"C", nullptr, B};
    B->next = C;
    Node* D = new Node{"D", nullptr, C};
    C->next = D;

    traverseForward(A);
    return 0;
}
```

Q 8: There are two teams named Dalta Gang and Malta Gang. Dalta Gang has 4 members, and each member has 2 Gullaks (piggy banks) with some money stored in them. Malta Gang has 2 members, and each member has 3

Gullaks. Both gangs store their Gullak money values in a 2D array. Write a C++ program to:

- Display the stored data in matrix form.
- To multiply Dalta Gang matrix with Malta Gang Matrix

```
#include <iostream>

using namespace std;


int main() {

    int dalta[4][2] = {

        {10, 20},

        {5, 15},

        {8, 12},

        {7, 13}

    };


    int malta[2][3] = {

        {2, 4, 6},

        {1, 3, 5}

    };


    int result[4][3] = {0};


    // Display Dalta Gang matrix

    cout << "Dalta Gang Matrix (4x2):\n";

    for (int i = 0; i < 4; i++) {
```

```
    for (int j = 0; j < 2; j++)
        cout << dalta[i][j] << "\t";
    cout << endl;
}

// Display Malta Gang matrix
cout << "\nMalta Gang Matrix (2x3):\n";
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++)
        cout << malta[i][j] << "\t";
    cout << endl;
}

// Matrix multiplication (4x2 × 2x3 = 4x3)
for (int i = 0; i < 4; i++) {
    for (int j = 0; j < 3; j++) {
        result[i][j] = 0;
        for (int k = 0; k < 2; k++) {
            result[i][j] += dalta[i][k] * malta[k][j];
        }
    }
}

// Display result matrix
cout << "\nResultant Matrix (4x3):\n";
```

```
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 3; j++)
            cout << result[i][j] << "\t";
        cout << endl;
    }

    return 0;
}
```

SECTION -B

Q 1: To store the names of family members, an expert suggests organizing the data in a way that allows efficient searching, traversal, and insertion of new members. For this purpose, use a **Binary Search Tree (BST)** to store the names of family members, starting with the letters:

<Q, S, R, T, M, A, B, P, N>

Write a C++ program to Create a **Binary Search Tree (BST)** using the given names and find and display the **successor** of the family member whose name starts with **M**.

```
#include <iostream>
using namespace std;

struct Node {
    string data;
    Node* left;
    Node* right;
};
```

```
Node* createNode(string value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->left = newNode->right = nullptr;
    return newNode;
}

Node* insert(Node* root, string value) {
    if (root == nullptr) return createNode(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);
    return root;
}

Node* minValueNode(Node* node) {
    Node* current = node;
    while (current && current->left != nullptr)
        current = current->left;
    return current;
}

Node* search(Node* root, string key) {
```

```
    if (root == nullptr || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}

void inorder(Node* root) {
    if (root != nullptr) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

void preorder(Node* root) {
    if (root != nullptr) {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root) {
    if (root != nullptr) {
        postorder(root->left);
        postorder(root->right);
```

```
            cout << root->data << " ";
        }
    }
```

Q 2: Implement the In-Order, Pre- Order and Post-Order traversal of Binary search tree with help of C++ Program.

```
#include <iostream>
using namespace std;

// Define the node structure
struct Node {
    int data;
    Node* left;
    Node* right;
};

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = newNode->right = nullptr;
    return newNode;
}

// Function to insert a value into the BST
Node* insert(Node* root, int value) {
```

```
    if (root == nullptr) {
        return createNode(value);
    }
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    return root;
}

// In-order traversal (L → Root → R)
void inorder(Node* root) {
    if (root != nullptr) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

// Pre-order traversal (Root → L → R)
void preorder(Node* root) {
    if (root != nullptr) {
        cout << root->data << " ";
        preorder(root->left);
```

```
        preorder(root->right);
    }
}

// Post-order traversal (L → R → Root)
void postorder(Node* root) {
    if (root != nullptr) {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

int main() {
    Node* root = nullptr;

    // Insert sample data into BST
    int values[] = {50, 30, 70, 20, 40, 60, 80};
    for (int val : values) {
        root = insert(root, val);
    }

    cout << "In-Order Traversal: ";
    inorder(root);
    cout << endl;
```

```
    cout << "Pre-Order Traversal: ";
    preorder(root);
    cout << endl;

    cout << "Post-Order Traversal: ";
    postorder(root);
    cout << endl;

    return 0;
}
```

Q 3: Write a C++ program to search an element in a given binary search Tree.

```
#include <iostream>
using namespace std;

// Define Node structure
struct Node {
    int data;
    Node* left;
    Node* right;
};

// Create new node
Node* createNode(int value) {
```

```
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = newNode->right = nullptr;
    return newNode;
}

// Insert into BST
Node* insert(Node* root, int value) {
    if (root == nullptr) return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

// Search function
Node* search(Node* root, int key) {
    if (root == nullptr || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);
```

```
    return search(root->right, key);
}

int main() {
    Node* root = nullptr;

    // Insert sample data into BST
    int values[] = {50, 30, 70, 20, 40, 60, 80};
    for (int val : values) {
        root = insert(root, val);
    }

    int key;
    cout << "Enter the element to search: ";
    cin >> key;

    Node* result = search(root, key);

    if (result != nullptr)
        cout << "Element " << key << " found in BST." << endl;
    else
        cout << "Element " << key << " not found in BST." << endl;

    return 0;
```

}

Q 4: In a university, the roll numbers of newly admitted students are: 45, 12, 78, 34, 23, 89, 67, 11, 90, 54

The administration wants to store these roll numbers in a way that allows **fast searching, insertion, and retrieval in ascending order**. For efficiency, they decide to apply a **Binary Search Tree (BST)**.

Write a C++ program to construct a **Binary Search Tree** using the above roll numbers and perform an **in-order traversal** to display them in ascending order.

```
#include <iostream>

using namespace std;


// Define the structure of a Node

struct Node {

    int data;

    Node* left;

    Node* right;

};


// Function to create a new node

Node* createNode(int value) {

    Node* newNode = new Node();

    newNode->data = value;

    newNode->left = newNode->right = nullptr;

    return newNode;
```

```
}

// Function to insert data into the BST
Node* insert(Node* root, int value) {
    if (root == nullptr) {
        return createNode(value);
    }

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

// Function for in-order traversal (Left -> Root -> Right)
void inorderTraversal(Node* root) {
    if (root != nullptr) {
        inorderTraversal(root->left);
        cout << root->data << " ";
        inorderTraversal(root->right);
    }
}
```

```
int main() {
    Node* root = nullptr;

    // Roll numbers to be inserted in BST
    int rollNumbers[] = {45, 12, 78, 34, 23, 89, 67, 11, 90, 54};

    // Construct the BST
    for (int i = 0; i < 10; i++) {
        root = insert(root, rollNumbers[i]);
    }

    cout << "Roll numbers in ascending order (In-Order Traversal):\n";
    inorderTraversal(root);
    cout << endl;

    return 0;
}
```

Q 5: In a university database, student roll numbers are stored using a **Binary Search Tree (BST)** to allow efficient searching, insertion, and deletion. The roll numbers are: 50, 30, 70, 20, 40, 60, 80. The administrator now wants to **delete a student record** from the BST. Write a C++ program to delete a node (student roll number) entered by the user.

```
#include <iostream>
using namespace std;

// Structure of a BST Node
struct Node {
    int data;
    Node* left;
    Node* right;
};

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->left = newNode->right = nullptr;
    return newNode;
}

// Function to insert a node into BST
Node* insert(Node* root, int value) {
    if (root == nullptr) return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
```

```
        root->right = insert(root->right, value);

    return root;
}

// Function to find minimum value node in a BST (used in deletion)
Node* findMin(Node* root) {
    while (root && root->left != nullptr)
        root = root->left;
    return root;
}

// Function to delete a node from BST
Node* deleteNode(Node* root, int key) {
    if (root == nullptr) return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Node to be deleted found

        // Case 1: No child
        if (root->left == nullptr && root->right == nullptr) {
```

```
            delete root;
            return nullptr;
        }
        // Case 2: One child
        else if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        // Case 3: Two children
        Node* temp = findMin(root->right);  // Get inorder successor
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// In-order Traversal to display BST
void inorderTraversal(Node* root) {
    if (root != nullptr) {
```

```
        inorderTraversal(root->left);
        cout << root->data << " ";
        inorderTraversal(root->right);
    }
}

int main() {
    Node* root = nullptr;
    int rollNumbers[] = {50, 30, 70, 20, 40, 60, 80};

    // Construct BST
    for (int i = 0; i < 7; i++) {
        root = insert(root, rollNumbers[i]);
    }

    cout << "Original Roll Numbers (In-Order Traversal):\n";
    inorderTraversal(root);
    cout << endl;

    int key;
    cout << "\nEnter roll number to delete: ";
    cin >> key;

    root = deleteNode(root, key);
```

```
    cout << "\nRoll Numbers after deletion (In-Order Traversal):\n";
    inorderTraversal(root);
    cout << endl;

    return 0;
}
```

Q 6: Design and implement a **family tree hierarchy** using a **Binary Search Tree (BST)**. The family tree should allow efficient storage, retrieval, and manipulation of information related to individuals and their relationships within the family.

Write a C++ program to:

1. Insert family members into the BST (based on their names).
2. Perform in-order, pre-order, and post-order traversals to display the hierarchy.
3. Search for a particular family member by name.

```
#include <iostream>
#include <string>
using namespace std;

// Node structure for BST
struct Node {
    string name;
    Node* left;
    Node* right;
};

// Function to create a new node
Node* createNode(string name) {
    Node* newNode = new Node();
    newNode->name = name;
    newNode->left = newNode->right = nullptr;
```

```
    return newNode;
}

// Function to insert a family member into BST (based on name)
Node* insert(Node* root, string name) {
    if (root == nullptr)
        return createNode(name);

    if (name < root->name)
        root->left = insert(root->left, name);
    else if (name > root->name)
        root->right = insert(root->right, name);

    return root;
}

// In-Order Traversal (Left -> Root -> Right)
void inorderTraversal(Node* root) {
    if (root != nullptr) {
        inorderTraversal(root->left);
        cout << root->name << " ";
        inorderTraversal(root->right);
    }
}
```

```
// Pre-Order Traversal (Root -> Left -> Right)
void preorderTraversal(Node* root) {
    if (root != nullptr) {
        cout << root->name << " ";
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

// Post-Order Traversal (Left -> Right -> Root)
void postorderTraversal(Node* root) {
    if (root != nullptr) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        cout << root->name << " ";
    }
}

// Search for a family member
bool search(Node* root, string key) {
    if (root == nullptr)
        return false;

    if (root->name == key)
        return true;
```

```
    else if (key < root->name)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main() {
    Node* root = nullptr;
    int choice;
    string name;

    do {
        cout << "\n--- Family Tree Menu ---\n";
        cout << "1. Insert Family Member\n";
        cout << "2. Display In-Order Traversal\n";
        cout << "3. Display Pre-Order Traversal\n";
        cout << "4. Display Post-Order Traversal\n";
        cout << "5. Search Family Member\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter member name: ";
```

```
        cin >> name;
        root = insert(root, name);
        cout << "Member inserted successfully.\n";
        break;

    case 2:
        cout << "Family Tree (In-Order): ";
        inorderTraversal(root);
        cout << endl;
        break;

    case 3:
        cout << "Family Tree (Pre-Order): ";
        preorderTraversal(root);
        cout << endl;
        break;

    case 4:
        cout << "Family Tree (Post-Order): ";
        postorderTraversal(root);
        cout << endl;
        break;

    case 5:
        cout << "Enter name to search: ";
```

```
            cin >> name;
            if (search(root, name))
                cout << name << " is present in the family tree.\n";
            else
                cout << name << " not found in the family tree.\n";
            break;

        case 6:
            cout << "Exiting...\n";
            break;

        default:
            cout << "Invalid choice! Try again.\n";
        }

    } while (choice != 6);

    return 0;
}
```